

The Growth Layer

One admin-controlled layer for coupons and discounts, and a beta waitlist that remembers who it passed over — so the people we skip this round go first in the next one.

In one paragraph. We have more people wanting into the beta than we have places. Today there is nothing in the product that records who asked, who we let in, or who we turned away — so every round we would pick from scratch, and the founder who has been waiting since day one can be skipped forever without anyone noticing. This proposal adds a queue with a memory, a two-step selection flow that cannot email 200 people by accident, and a coupon system the admin panel controls so a beta invite can carry a real offer.

STATUS

Proposal — for review

SURFACE

Admin Panel + backend

DATE

4 September 2026

Why we need this

Two problems that look unrelated, solved by one layer — because they meet at exactly one point: the invite email.

Problem 1 — the waitlist has no memory

Say 200 founders register for the beta and we can support 100. We pick 100. Next month we open another 100 places. Who goes first?

Right now the honest answer is *whoever happens to be at the top of a spreadsheet*. Nothing records that a founder was already passed over once. A person can be skipped three rounds running while somebody who signed up yesterday gets in ahead of them, and we would never find out — they just quietly stop caring about us.

Problem 2 — offers live in people's heads

There is no coupon or discount concept anywhere in the codebase. Any offer we make today is a promise made in an email that no part of the system can check, count, or expire.

That means we cannot cap how many people claim it, cannot stop one person using it five times, cannot see how many were redeemed, and cannot turn it off without emailing everyone again.

What "solved" looks like

TODAY	AFTER THIS CHANGE
Beta signups live in a spreadsheet or an inbox.	One list in the admin panel, with a status per person.
Selecting 100 people means copying IDs by hand.	One button picks the correct next 100, in the correct order. You can still override it person by person.
Nobody is told they were not selected.	Everyone hears something, and the people we skipped are told exactly where they now stand.
Being skipped has no consequence.	Being skipped is what moves you to the front of the queue.
A discount is a sentence in an email.	A discount is a code with a cap, a window, and a record of every claim.

THE THING WORTH ARGUING ABOUT

The whole design turns on one decision: **we email the people we did *not* pick**. It is more work, and it invites a reply we may not want. We think it is right — a silent waitlist is indistinguishable from a rejection, and a founder who is told "you are next, and here is why" stays warm. The behaviour is a per-round toggle, so we can turn it off if the first round proves us wrong.

The queue, and how it remembers

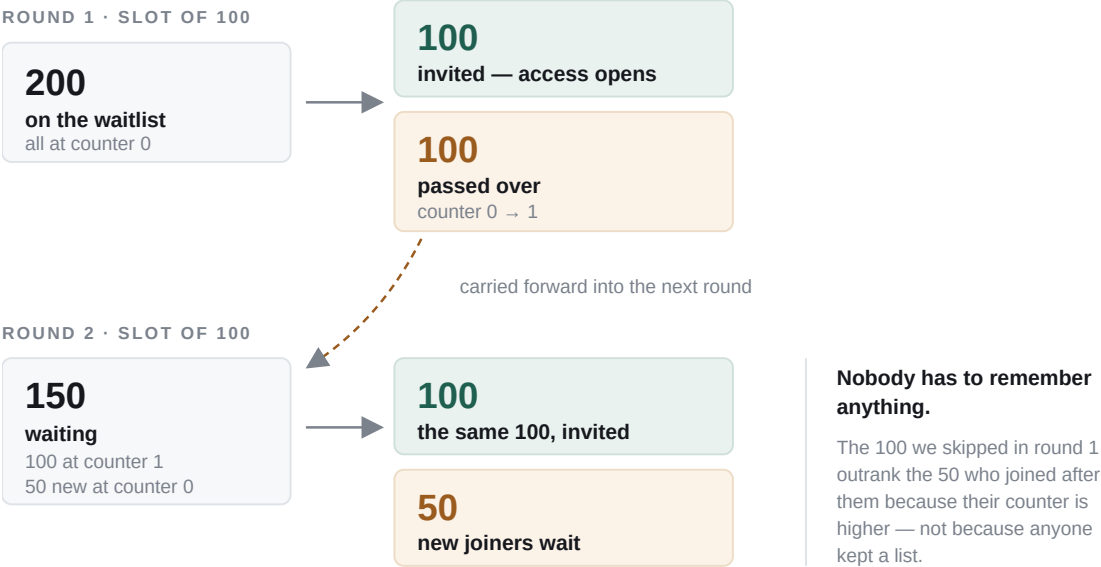
Every founder on the waitlist carries a counter. Each release adds one to that counter for everyone still waiting. The queue sorts by it. That is the entire mechanism.

Concretely, the order we invite from is:

```
ORDER BY times_deferred DESC, priority DESC, joined_at ASC
```

READ LEFT TO RIGHT	WHAT IT MEANS IN THE ROOM
times_deferred , highest first	Whoever we have passed over most times goes first. Being skipped is the thing that promotes you.
priority , highest first	A deliberate bump for someone we genuinely want early — an investor referral, a design partner. Set by hand, recorded in the audit log.
joined_at , oldest first	Among equals, first come first served. This is the tiebreak, not the main rule.

A worked round, with the numbers from the brief



Two consecutive rounds. The counter is what makes round 2 fair; no manual bookkeeping is involved, and the order is the same whoever clicks the button.

WHY A STORED COUNTER RATHER THAN WORKING IT OUT LATER

We could in principle reconstruct "who did we skip" by joining cohort membership against release timestamps. It would be correct in theory and wrong the first time anyone edits a round by hand or a release half-fails. The counter is one integer that is true by construction.

What running a round actually looks like

Five steps in the admin panel. The important design decision is that **picking people and emailing them are two separate buttons**.

- 1 Open a round**

Give it a name ("Beta round 2") and a size (100). Optionally attach a coupon code that every invite in this round will carry. If the code does not exist or has expired, we refuse right here — better than discovering it in 100 emails quoting an offer that checkout will reject.
- 2 Pick the people**

Press *auto-pick 100* and the queue fills the round in the correct order. Or tick people by hand. Or paste a list of IDs. All three end in the same place, and the panel shows you the list before anything happens.
- 3 Review, and change your mind freely**

Selection sends nothing. You can re-pick as many times as you like; the previous staging is replaced, not added to, so a corrected list can never quietly overshoot the slot size.
- 4 Release**

This is the irreversible one, and it asks for explicit confirmation. In one operation: the selected become *invited*, everyone still waiting has their counter incremented, and both groups have an email queued.
- 5 Watch the mail go out**

The panel shows sent / failed / not-yet-sent per person. A bounced address can be re-queued individually. Nothing is ever sent twice.

Why two steps and not one

Selection is reviewable and reversible. Release mails every founder on the list. Those have completely different consequences if you click them by mistake, so they are two buttons. Merging them means the first slip of the mouse emails 200 people.

Why release does not send the mail itself

Release records the decision and stops. A separate job drains the mail queue. If the server restarts halfway through 200 messages, the queue is still on disk and we simply drain it again — and because only *unsent* rows are picked up, re-running it can never double-send.

What the two groups actually receive

The second email is the one that matters. Telling 100 people they are in is easy; the 100 who are not are the ones who decide whether our waitlist reads as a queue or a lottery.

The 100 who are in

to: priya@example.com

You're in — your Ally beta access is open

Hi Priya,

You've been selected for the Ally beta (Beta round 2). Your access is open now.

Your code: **BETA20** (20% off). Apply it when you pick a plan.

Start here →

We read every piece of feedback in this phase, so tell us what breaks.

The GoXL Team

The 100 who are not

to: arun@example.com

You're next in line for the Ally beta

Hi Arun,

We've just opened this round of the Ally beta (Beta round 2) and your name wasn't in it. Places are limited, so this is about batch size, not about you.

You're now at the front of the queue for the next round. We invite from the top of that queue, oldest first, so you don't have to do anything to keep your place.

You'll hear from us when the next slot opens.

The GoXL Team

Why the second email says a number

Once someone has been passed over more than once, the wording changes to name it: *"You've been ahead of new signups for 2 rounds now, and you stay ahead until you're in."* That is a claim they can hold us to. "We'll be in touch" is not — and people can tell the difference immediately.

ONE THING TO DECIDE TOGETHER

If we run rounds weekly, the same founder gets a "you're next" email every week, which stops being reassuring and starts being annoying. The toggle is per round, so the safe default is: send it on the first few rounds, and if we go to a fast cadence, turn it off and move that information into the product instead.

What founders can see for themselves

A logged-in founder can check their own standing — status, when they joined, how many rounds they have been passed over. Nobody can see anybody else's, and an anonymous visitor sees nothing at all: a signup form that tells you whether an address is already registered is an email-enumeration tool, so joining the list gives the same response either way.

Coupons and discounts

A coupon here is a **rule**, not money. The layer owns three questions and deliberately nothing else.

1. What exists?

The set of codes, what each one is worth, who it applies to, and when it stops working.

2. May this person use it?

Checked on the server, every time, with a reason we can show the founder when the answer is no.

3. Who has claimed it?

A permanent ledger row per redemption. This is what makes caps enforceable and reporting possible.

Four kinds of discount

TYPE	VALUE MEANS	EXAMPLE
Percent	1–100, off the order total	20% off
Fixed	An absolute amount, stored in paise	Rs 500 off
Credits	Ally credits granted on redemption	1,500 bonus credits
Free days	Days of the plan given free	30 days free

Every value is an integer — money is never stored as a decimal fraction, so a discount cannot round differently in two places and produce two different invoices.

The controls on each code

CONTROL	WHAT IT STOPS
Total redemption cap	An offer meant for 100 people reaching 4,000 because it was shared publicly.
Per-founder cap	One person claiming the same discount every month.
Start / expiry window	A launch offer still being honoured a year later.
Plan restriction	A Pro-only offer being applied to Starter.
Active switch	Needing to email everyone to withdraw an offer.

What we are deliberately NOT building yet

There is no payment provider wired into this codebase. So this layer defines discounts and records claims — it does not change what anyone is charged. A "discount" that pretended to would be fiction. When checkout arrives it asks this layer *"is this code valid, and what is it worth?"* and applies the answer.

Two rules worth knowing

A code's value cannot be edited. A code somebody already redeemed at 20% must keep meaning 20% — silently re-pointing it would rewrite what past redemptions were worth. Retire it and issue a new one.

Validation fails closed. Unknown code, expired window, exhausted cap, wrong plan, plan not specified — all refused, each with its own reason.

The shape of the build

Four new tables, one new admin section, and one small public endpoint. Everything follows the pattern the admin panel already uses.

Tables

TABLE	HOLDS	THE COLUMN THAT MATTERS
beta_waitlist_entries	One row per person who wants in	times_deferred
beta_cohorts	One row per round: name, size, coupon, when it went out	slot_size
coupons	The discount rules	discount_type + value
coupon_redemptions	Every claim, permanently	(code, founder_id)

The email address is the identity on the waitlist and is unique — so signing up twice does not get you two places, and, more to the point, does not let someone reset their own deferral counter by resubmitting the form. The waitlist row survives account deletion as a queue position with no account attached.

New admin pages NEW

- **Beta Access** — the waitlist in queue order, the round builder, per-person email status, and a "who's next" preview you can look at before committing.
- **Coupons** — create, retire, and see redemption counts against caps.

Who is allowed to do what

ACTION	SUPPORT	ADMIN	SUPER ADMIN
See the waitlist, rounds and coupons	—	✓	✓
Create / retire a coupon	—	—	✓
Open a round, select, release	—	—	✓

Releasing a round mails every founder on the list at once and cannot be undone. That is why it sits at the top tier — blast radius, not seniority. An Admin who can reset one founder's diagnosis still should not be one mis-click from 200 emails.

Every write — creating a coupon, bumping someone's priority, releasing a round — is recorded in the existing admin audit trail with who did it, when, from where, and what changed.

Every write — creating a coupon, bumping someone's priority, releasing a round — is recorded in the existing admin audit trail with who did it, when, from where, and what changed.

Plan, and what we need from you

Four phases. The first three are this proposal; the fourth waits on something outside it.

PHASE	SCOPE	ROUGH SIZE
1 • Backend	Tables and migration, the queue logic, coupon validation and the redemption ledger, the admin API, tests	~2 days
2 • Admin UI	The two new panel pages — Beta Access and Coupons	~1 day
3 • Mail	Both templates, the durable send queue, per-person status and retries	~0.5 day
4 • Later	Hook coupon validation into checkout — blocked until a payment provider is wired in	Not now

How we would know it worked

- Nobody's `times_deferred` keeps climbing while newer signups get in ahead of them. If that ever happens, the queue is broken and the number will show it.
- A round can be released, and every person on the list ends up with a recorded outcome — sent, failed, or explicitly not notified. No silent gaps.
- A coupon's redemption count never exceeds its cap, under any sequence of clicks.

Three decisions we need from the team

QUESTION	WHY IT MATTERS
Do we email the deferred group every round, or only the first few?	Weekly rounds turn a reassuring email into a nagging one. The toggle exists either way; we need a default.
Does an unused invite expire?	If someone is invited and never signs in, that place is spent. An expiry would return it to the queue — but it also means telling someone their invite lapsed.
Does accepting an invite auto-redeem the attached coupon?	Auto-redeeming is smoother but burns the code for someone who never subscribes. Leaving it to checkout is honest but adds a step.

WHAT WE ARE ASKING FOR

Agreement on the queue rule — *skipped founders go first, always* — and answers to the three questions above. Everything else in this document is an implementation detail we can change later without anyone noticing.